BÀI 4: SPRING BOOT VALIDATION

MỤC TIÊU

Hiểu các annotation validation trong Spring Boot, xác thực dữ liệu để đảm bảo tính toàn vẹn của dữ liệu trong ứng dụng

Tích hợp Validation vào dự án Spring Boot

Thực hiện được việc kiểm lỗi trên cả front-end và back-end trong ứng dụng spring boot

NỘI DUNG

01

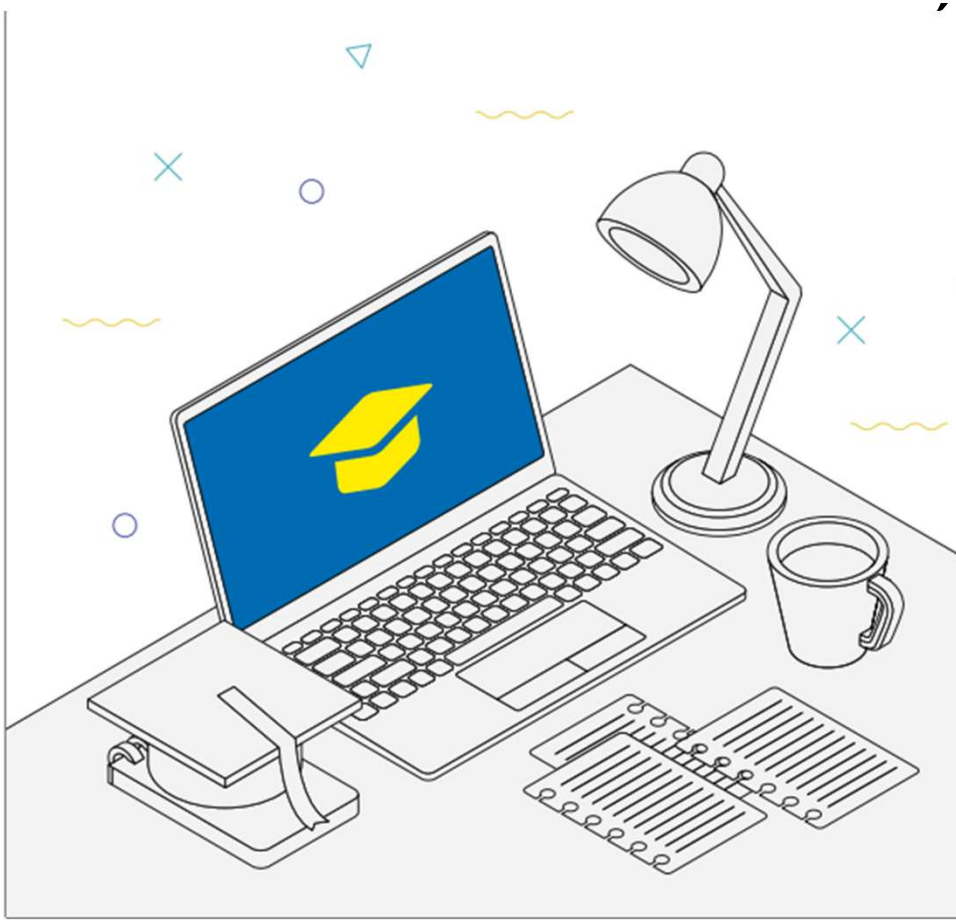
Giới thiệu về Spring Validation

02

Các Annotation xác thực dữ liệu

03

Xây dựng ứng dụng có Validation



BÀI 4

SPRING BOOT VALIDATION

Giới thiệu về Validation

GIỚI THIỆU VỀ VALIDATION

❑ Data Validation là gì?

- ✓ Data Validation là quá trình kiểm tra và đảm bảo tính hợp lệ của dữ liệu.
- ✓ Mục tiêu là đảm bảo dữ liệu đáp ứng các quy tắc nhất định trước khi được lưu trữ hoặc xử lý.
- ✓ Giúp ngăn ngừa lỗi, cải thiện chất lượng dữ liệu và bảo mật hệ thống.

  - Ví dụ

Name:		*Customer name should be between 2 and 30 characters long*
Email:	abc	*Email address is not valid*
Age:		*Customer age should be in years*
Gender:	Select Gender	*This is a required field*
Birthday:	2014/12/12	*birthday is of invalid format*
Phone:	sdsd	*Invalid format, valid formats are 1234567890, 123-456-7890 x1234*

Save Customer

GIỚI THIỆU VỀ VALIDATION

❑ Thực hiện Validation trong dự án

Validation phía Client (Client-side Validation): cải thiện trải nghiệm người dùng bằng cách kiểm tra dữ liệu trước khi gửi lên server

```
<form id="userForm">
    <input type="email" id="email" name="email" required>
    <input type="password" id="password" name="password" minlength="8" required>
    <button type="submit">Submit</button>
</form>
<script>
    document.getElementById("userForm").addEventListener("submit", function(event) {
        var email = document.getElementById("email").value;
        var password = document.getElementById("password").value;
        if (email === "" || password.length < 8) {
            alert("Email or Password is invalid");
            event.preventDefault();
        }
    });
</script>
```

GIỚI THIỆU VỀ VALIDATION

❑ Thực hiện Validation trong dự án

✓ Validation phía Server (Server-side Validation): bảo vệ dữ liệu và đảm bảo tính toàn vẹn của hệ thống. Dù người dùng có thể tắt JavaScript hoặc gửi dữ liệu không hợp lệ từ phía client, ứng dụng vẫn bảo vệ được bằng cách kiểm tra lại dữ liệu trên server.

Cách thực hiện trong Spring Boot:

- Sử dụng các **annotations** Validation trên các lớp model
- Kích hoạt validate ở **Controller**

```java
@Data
@NoArgsConstructor
public class Book {
    private Long id;
    @NotBlank(message = "Tiêu đề không được để trống")
    @Size(min = 2, max = 100, message = "Tiêu đề phải có độ dài từ 2
    private String title;

    @NotBlank(message = "Tác giả không được để trống")
    @Size(max = 50, message = "Tên tác giả không được vượt quá 50 ký
    private String author;
}
```

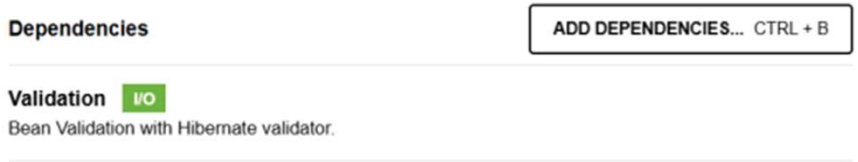
```java
@PostMapping("/add")
public String addBookForm(@Valid @ModelAttribute Book book, BindingResult result) {
    if (result.hasErrors()) {
        return "add-book";
    }
    bookService.addBook(book);
    return "redirect:/books";
}
```

GIỚI THIỆU VỀ VALIDATION

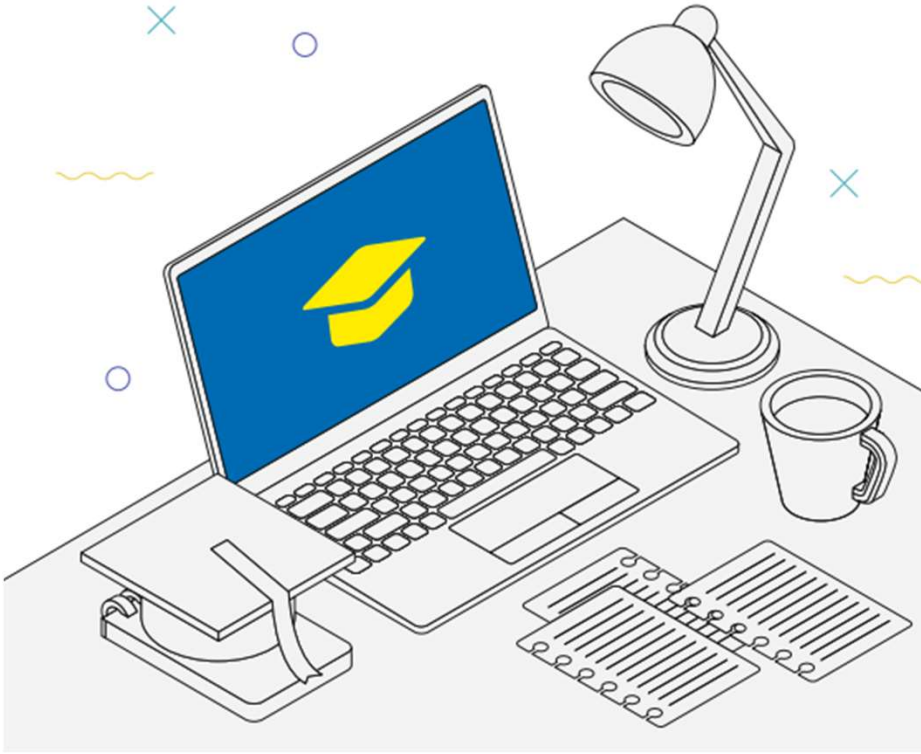
❑ Tích hợp Spring Validation

✓ Bổ sung thư viện **Validation** vào **pom.xml** của dự án Maven. Chạy lại Maven để tải các dependencies về.

```
<dependency>
<groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-
validation</artifactId>
</dependency>
```



Spring Validation là một phần quan trọng trong Spring Framework giúp kiểm tra và đảm bảo tính hợp lệ của dữ liệu đầu vào, đặc biệt khi xử lý các yêu cầu HTTP từ người dùng (ví dụ như form submission (gửi dữ liệu từ form HTML lên server để xử lý) hoặc từ các yêu cầu REST API).



BÀI 4

SPRING BOOT VALIDATION

Các annotation xác thực dữ liệu

CÁC ANNOTATION XÁC THỰC DỮ LIỆU

- Các annotation phổ biến trong Spring Validation

Annotation	Mô tả
@Valid	Được sử dụng trong controller để xác thực trên request body hoặc tham số phương thức
@NotNull	Đảm bảo không được phép null
@NotEmpty	Đảm bảo không được phép null và không được rỗng (áp dụng cho String, collection)
@NotBlank	Đảm bảo String không null, không rỗng và không chứa khoảng trắng
@Size	Định nghĩa kích thước của collection, array hoặc String. Ví dụ: @**Size**(min = 2, max = 30)
@Pattern	Xác thực theo biểu thức chính quy. Ví dụ: @**Pattern**(regexp = '^[a-zA-Z]+$')
@Min/@Max	Đảm bảo giá trị số nằm trong phạm vi quy định

CÁC ANNOTATION XÁC THỰC DỮ LIỆU

❑ Sử dụng Validation trong Spring Boot

- ✓ **Entity Layer:** Đặt các annotation trực tiếp trong các lớp Entity để tự động thực hiện validation khi nhận dữ liệu.
- ✓ **Controller Layer**:

@Valid để xác thực đối tượng được gửi từ client

@RestControllerAdvice giúp bạn xử lý lỗi từ validation và trả về phản hồi phù hợp cho người dùng

Sử dụng **@ExceptionHandler** để xử lý các lỗi validation toàn cục trong ứng dụng

Lớp **BindingResult** được sử dụng để kiểm tra kết quả validation. Nếu có lỗi, có thể xử lý và trả về thông báo lỗi thích hợp

BÀI 4

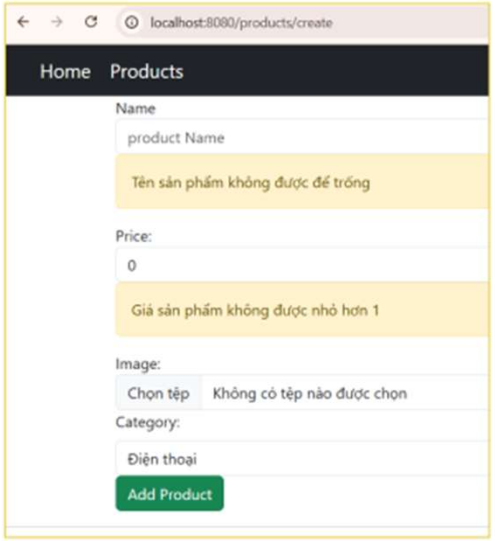
SPRING BOOT VALIDATION

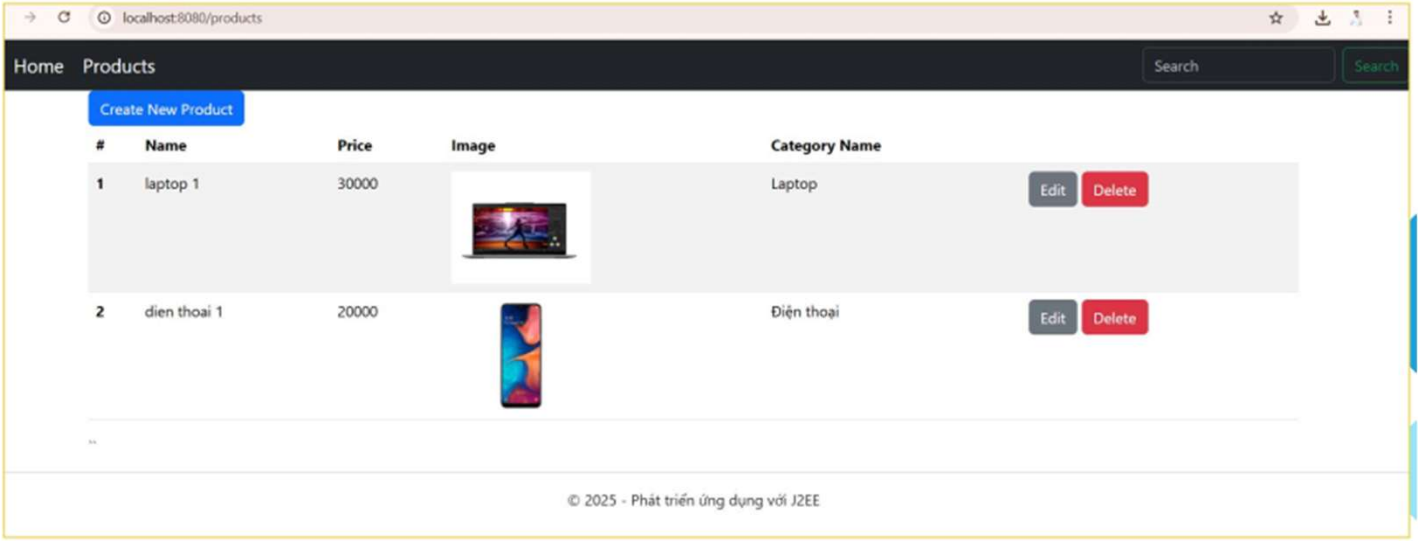
Xây dựng ứng dụng với Validation

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

Viết ứng dụng cho phép quản lý sản phẩm (mã sản phẩm, tên sản phẩm, giá, hình ảnh, loại danh mục). Giả sử có 2 danh mục "điện thoại" "laptop" được thể hiện ở dropdownlist tương ứng

- Khi thêm sản phẩm: thực hiện validation, upload hình ảnh
  - Tên sản phẩm không được để trống
  - Tên hình ảnh không quá 200 kí tự
  - Giá sản phẩm không được để trống và cho phép nhập từ 1 – 9999999
- Thực hiện lấy danh sách, thay đổi, xóa

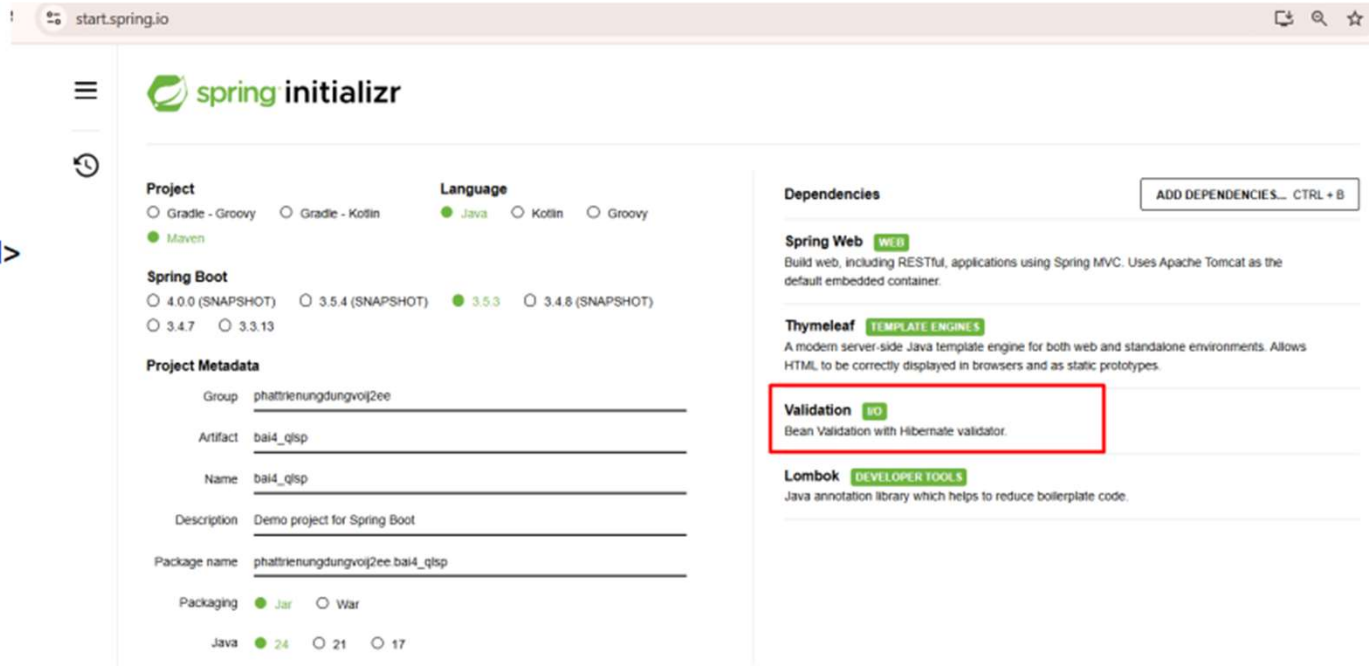




XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

- Tạo dự án từ spring initializr
  - ✓ Các dependencies: Spring Web, Thymeleaf, Validation, Lombok
  - ✓ Bổ sung thêm **layout-dialect** cho dự án để sử dụng Layout Page, Content Page

```
<dependency>
<groupId>nz.net.ultraq.thymeleaf</groupId>
  <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```



XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

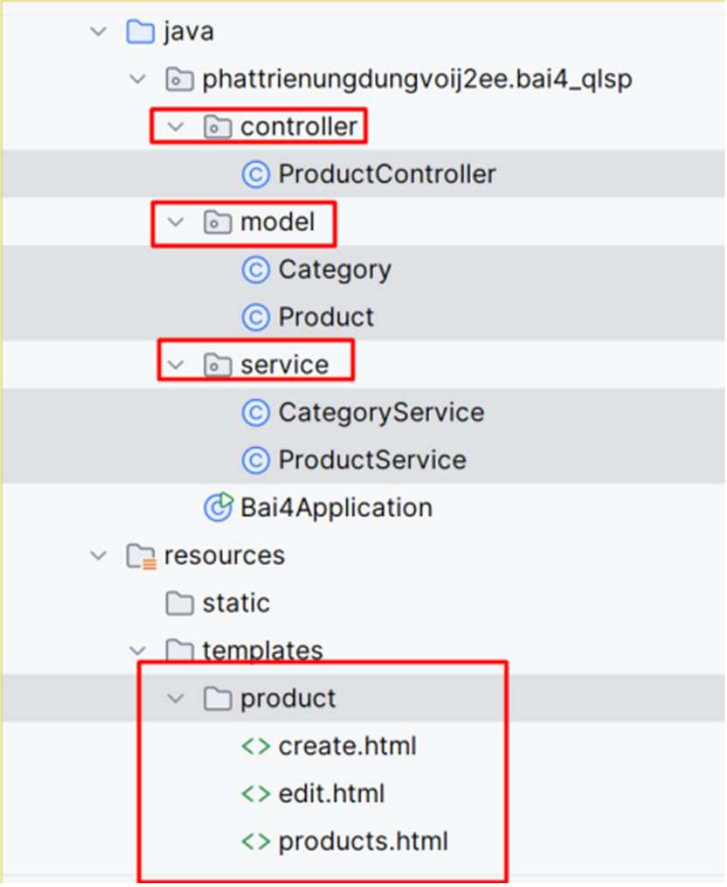
- B1: Xây dựng kiến trúc: model – service – controller
  - ✓ Thêm các lớp java trong kiến trúc **model-service-controller**
  - ✓ Tạo các templates cho giao diện

    _layout.html: Layout Page

    products.html: trang danh sách sản phẩm

    create.html: trang tạo mới sản phẩm

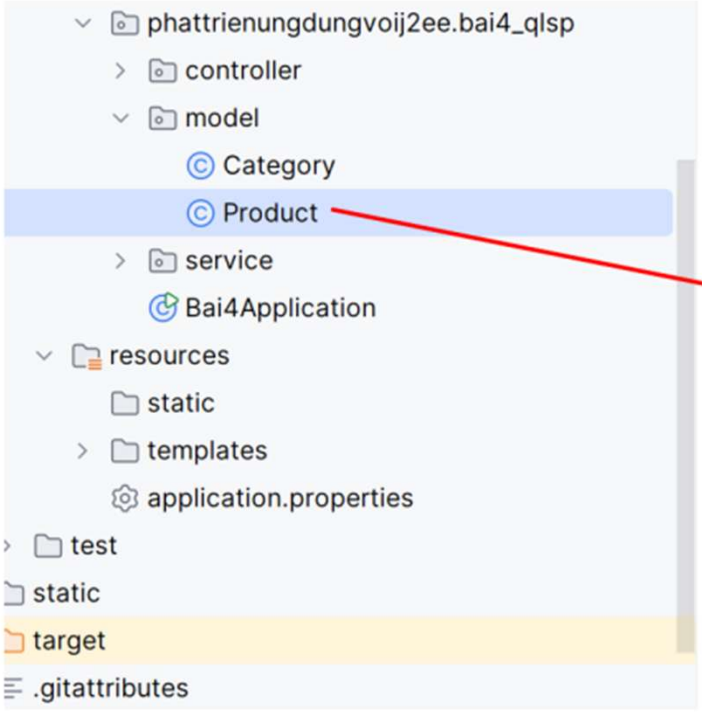
    edit.html: trang chỉnh sửa thông tin sản phẩm



XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

❑ B1: Xây dựng kiến trúc: **model** – service – controller

✓ Thực hiện **Product**




```java
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.Setter;
import org.hibernate.validator.constraints.Length;
@Getter
@Setter
@AllArgsConstructor
public class Product {
    private int id;
    @NotBlank(message = "Tên sản phẩm không được để trống")
    private String name;

    @Length(min= 0,max = 200, message = "Tên hình ảnh không quá 200 kí tự")
    private String image;

    @NotNull(message = "Giá sản phẩm không được để trống")
    @Min(value = 1, message = "Giá sản phẩm không được nhỏ hơn 1")
    @Max(value = 9999999, message = "Giá sản phẩm không được lớn hơn 9999999")
    private long price;

    private Category category;
}
```

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

- B1: Xây dựng kiến trúc: **model** – service – controller
  - ✓ Thực hiện **Category**

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

❑ B1: Xây dựng kiến trúc: model – service – controller

```java
@Service
public class ProductService {
    List<Product> listProduct = new ArrayList<>();
    public List<Product> getAll() {
        return listProduct;
    }
    public Product get(int id) {
        return listProduct.stream().filter(p->p.getId()
== id).findFirst().orElse(null);
    }
    public void add(Product newProduct) {
        int maxId =
listProduct.stream().mapToInt(Product::getId).max().orEls
e(0);
        newProduct.setId(maxId+1);
        listProduct.add(newProduct);
    }
    public void update(Product editProduct)
    {
        Product find = get(editProduct.getId());
        if(find≠ null) {
            find.setPrice(editProduct.getPrice());
            find.setName(editProduct.getName());
            if(editProduct.getImage()≠ null)
                find.setImage(editProduct.getImage());
        }
    }
```

✓ Thực hiện **ProductService**

```java
public void updateImage(Product newProduct, MultipartFile imageProduct)
    {
        String contentType = imageProduct.getContentType();
        if (contentType ≠ null && IllegalArgumentException("Tệp tải lên
không phải là hình ảnh!");
        }
        if (!imageProduct.isEmpty()) {
            try
            {
                Path dirImages = Paths.get("static/images");
                if (!Files.exists(dirImages)) {
                    Files.createDirectories(dirImages);
                }
                String newFileName = UUID.randomUUID() + "_" +
imageProduct.getOriginalFilename();
                Path pathFileUpload = dirImages.resolve(newFileName);
                Files.copy(imageProduct.getInputStream(), pathFileUpload,
StandardCopyOption.REPLACE_EXISTING);
                newProduct.setImage(newFileName);
            }
            catch (IOException e) {
                e.printStackTrace(); // Handle the exception appropriately
            }
        }
    }
}
```

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

- B1: Xây dựng kiến trúc: model – service – controller

```java
@Controller
@RequestMapping("/products")
public class ProductController {
    @Autowired
    private ProductService productService;
    @Autowired
    private CategoryService categoryService;
    @GetMapping()
    public String Index(Model model) {
        model.addAttribute("listproduct", productService.getAll());
        return "product/products";
    }
    @GetMapping("/create")
    public String Create(Model model) {
        model.addAttribute("product", new Product());
        model.addAttribute("categories", categoryService.getAll());
        return "product/create";
    }
    @PostMapping("/create")
    public String Create(@Valid Product newProduct,BindingResult result,
                         @RequestParam("category.id") int categoryId,
     @RequestParam("imageProduct") MultipartFile imageProduct,Model model) {
        if (result.hasErrors()) {
            model.addAttribute("product", newProduct);
            model.addAttribute("categories", categoryService.getAll());
            return "product/create";
        }
        productService.updateImage(newProduct, imageProduct); // Xử lý ảnh
        Category selectedCategory = categoryService.get(categoryId);
        newProduct.setCategory(selectedCategory);
        productService.add(newProduct);
        return "redirect:/products";
    }
```

```java
@GetMapping("/edit/{id}")
public String Edit(@PathVariable int id, Model model) {
    Product find = productService.get(id);
    if (find == null) {
        return "error/404"; // Trang lỗi tùy chỉnh
    }
    model.addAttribute("product", find);
    model.addAttribute("categories", categoryService.getAll());
    return "product/edit";
}

@PostMapping("/edit")
public String Edit(@Valid Product editProduct,
                   BindingResult result,
                   @RequestParam("imageProduct") MultipartFile
imageProduct,
                   Model model) {
    if (result.hasErrors()) {
        model.addAttribute("product", editProduct);
        model.addAttribute("categories",  categoryService.getAll());
        return "product/edit";
    }
    if (imageProduct != null && !imageProduct.isEmpty()) {
        productService.updateImage(editProduct, imageProduct); //
Cập nhật ảnh nếu có
    }
    productService.update(editProduct); // Cập nhật sản phẩm
    return "redirect:/products";
}
```

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

❑ B2: Thiết kế templates ứng dụng

✓ Trang layout: **_layout.html**

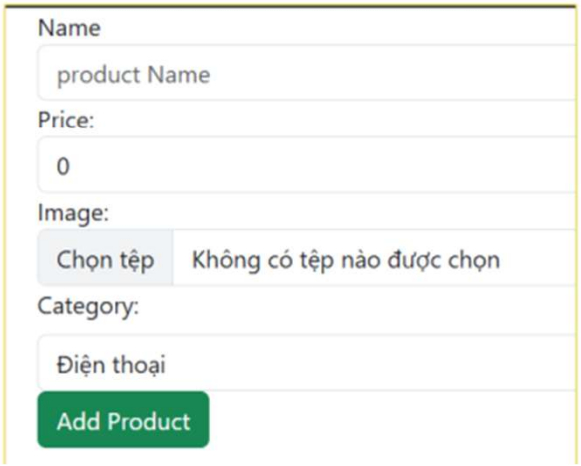
```
<html lang="en"
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"xmlns:th="http://www.w3.org/1999/xhtml">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
    <meta name="description" content="BigSchool - Home page with courses and search
functionality">
    <title>Demo Products</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
          rel="stylesheet"
          integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
          crossorigin="anonymous">
</head>
<body>
<nav class="navbar bg-dark border-bottom border-body" data-bs-theme="dark">
    <div style="padding-left: 40px">
        <a class="navbar-brand" th:href="@{/products}">Home</a>
        <a class="navbar-brand" th:href="@{/products/create}">Products</a>
    </div>
    <form class="d-flex" role="search" th:action="@{/home/search}" th:method="get">
        <input class="form-control me-2" type="search" placeholder="Search" aria-label="Search">
        <button class="btn btn-outline-success" type="submit">Search</button>
    </form>
</nav>
<div layout:fragment="content" class="container body-content">
</div>
<div class="card-footer text-center">
    <hr/>
    <p>&copy; 2025 - Phát triển ứng dụng với J2EE</p>
</div>
</body>
</html>
```

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

❑ B2: Thiết kế templates ứng dụng

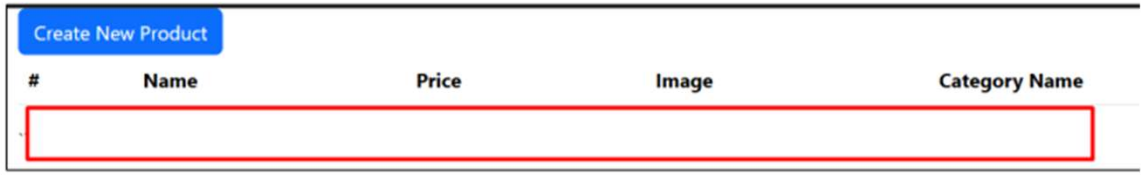
✓ Trang content: product/**create.html**

```
<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{_layout}">
<head>
    <meta charset="UTF-8">
    <title>Create product</title>
</head>
<body>
<div layout:fragment="content" class="container body-content">
    <form th:action="@{/products/create}" th:object="${product}"
method="post" class="form" enctype="multipart/form-data">
        <div class="form-group">
            <label for="name">Name</label>
            <input class="form-control" type="text" th:field="*{name}"
id="name"  placeholder="product Name">
            <div class="alert alert-warning"
th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></div>
        </div>
        <div class="form-group">
            <label for="price">Price:</label>
            <input class="form-control" type="number" th:field="*{price}"
id="price" placeholder="price">
            <div class="alert alert-warning"
th:if="${#fields.hasErrors('price')}" th:errors="*{price}"></div>
        </div>
 <div class="form-group">
            <label for="image">Image:</label>
            <input class="form-control" type="file" id="image" name="imageProduct"
accept="image/png,image/jpeg">
            <div class="alert alert-warning" th:if="${#fields.hasErrors('image')}"
th:errors="*{image}"></div>
        </div>
        <div class="form-group">
            <label for="category" class="form-label">Category:</label>
            <select th:field="*{category.id}" class="form-control" id="category">
                <option th:each="category : ${categories}"
th:value="${category.id}" th:text="${category.name}"></option>
            </select>
            <div class="alert alert-warning"
th:if="${#fields.hasErrors('category')}" th:errors="*{category}"></div>
        </div>
        <input type="submit"  class="btn btn-success" value="Add Product">
    </form>
</div>
</body>
</html>
```



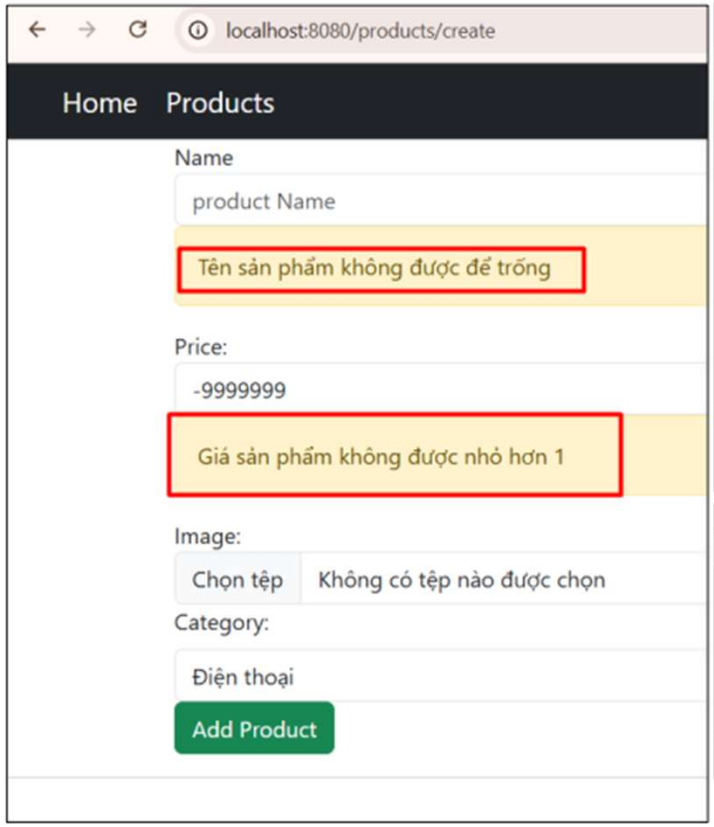
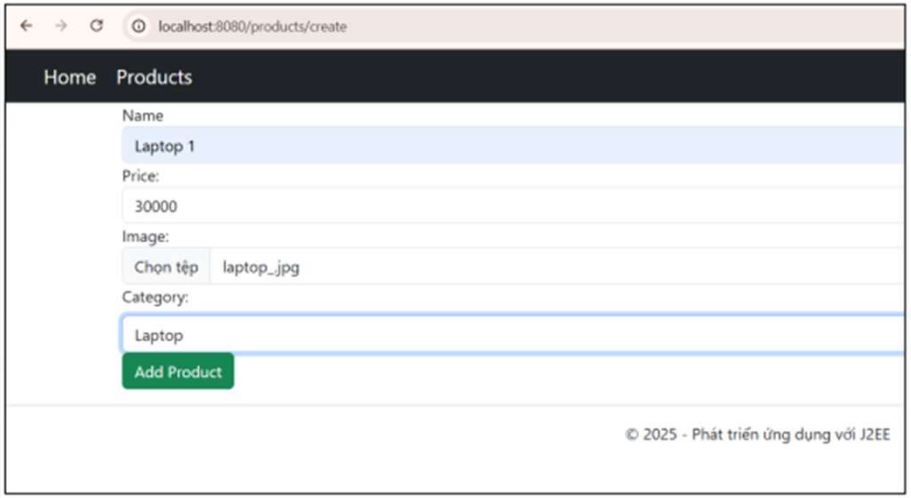
XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

- B2: Thiết kế templates ứng dụng
  - ✓ Trang content: product/**products.html**




```
<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{_layout}"
xmlns:custom="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="UTF-8">
    <title>List products</title>
</head>
<body>
<div layout:fragment="content" class="container body-content">
    <a th:href="@{products/create}" class="btn btn-
primary">Create New Product</a>
    <table class="table table-striped">
        <thead>
        <tr>
            <th scope="col">#</th>
            <th scope="col">Name</th>
            <th scope="col">Price</th>
            <th scope="col">Image</th>
            <th scope="col">Category Name</th>
            <th scope="col"></th>
        </tr>
        </thead>
```

```
<tbody>
        <tr th:each="product : ${listproduct}" >
            <th scope="row" th:text="${product.id}"></th>
            <td th:text="${product.name}"></td>
            <td th:text="${product.price}"></td>
            <td>
                <img height="120" width="150" th:src="@{/images/} +
${product.image}" alt="Product Image">
            </td>
            <td th:text="${product.category?.name ?: ''}"></td>
            <td>
                <a th:href="@{/products/edit/{id}(id=${product.id})}"
custom:linkMethod="post"  class="btn btn-secondary">Edit</a>
                <a th:href="@{/products/delete/{id}(id=${product.id})}"
custom:linkMethod="post" class="btn btn-danger">Delete</a>
            </td>
        </tr>
        </tbody>
    </table>``
</div>
</body>
</html>
```

XÂY DỰNG ỨNG DỤNG VỚI VALIDATION

Kết quả